Report
林歆祐
R14922159

Report Q1: Retriever & Reranker Tuning (5%)

Retriever Training Process (2.5%): Clearly describe how the training data is constructed

- The training data is constructed by the **load_pairs_for_mnrl** function in retriever.py. This function reads a JSONL file **(./data/train.txt)** and **generates (query, positive_passage) pairs**.
- Anchor (Query): The anchor is the query text. It is read from the **"rewrite" field of each JSON line**. The _normalize_query function automatically prefixes this text with "query: "
- Positive Sampling: Positive passages are explicitly defined in the data. The script reads the **"evidences"** list and a corresponding **"retrieval_labels"** list. Any evidence with a label of 1 is considered a positive passage. The bash script sets max_pos_per_query to 8, meaning a maximum of 8 positive passages are loaded for any single query. These passages are prefixed with "passage: " by the _normalize_passage function.
- Negative Sampling: The script uses an **in-batch negative strategy**. The load_pairs_for_mnrl function only loads positive pairs (e.g., InputExample(texts=[q, p])). The **MultipleNegativesRankingLoss implicitly treats all other passages within the same training batch as negative examples for a given query**.

Report Q1: Retriever & Reranker Tuning (5%)

Retriever Training Process (2.5%) (cont.) Specify and explain the loss function used for training:

The training uses **losses.MultipleNegativesRankingLoss** from the sentence-transformers library.

This loss function is an implementation of InfoNCE (Noise-Contrastive Estimation) designed for in-batch negatives. It is a contrastive loss that works as follows:

A batch is created containing **multiple (query, positive_passage) pairs**. For **a single query** in the batch, its corresponding passage is the **"positive" example**. **All other** passages in that same batch (which are positive examples for other queries) are treated as **"negative" examples** for the current query.

The loss function then optimizes the model to **maximize the similarity (e.g., cosine similarity)** between the query and its positive passage, while simultaneously minimizing the similarity between the query and all the in-batch negative passages. (show some similarity to InfoNCE loss)

$$\mathrm{Sim}(A_i, B_j) = \frac{A_i \cdot B_j}{\|A_i\| \|B_j\|}$$

$$\mathcal{L}_i = -\log \left(\frac{e^{\mathrm{Sim}(A_i, B_i)/\tau}}{\sum_{j=1}^{N} e^{\mathrm{Sim}(A_i, B_j)/\tau}} \right)$$

Report

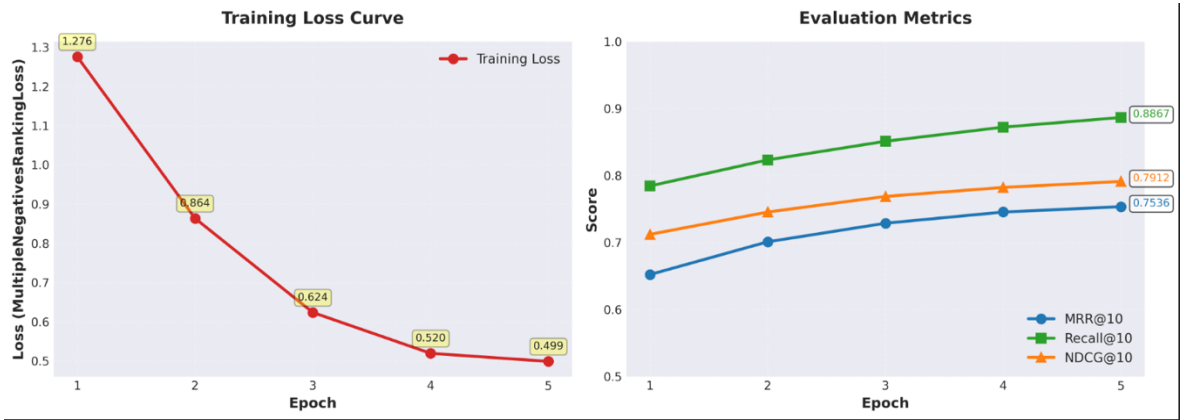
Q1: Retriever & Reranker Tuning (5%)

Retriever Training Process (2.5%) (cont.)

List the hyperparameters adopted in your experiments.

Epochs: 5
Batch Size: 64
Learning Rate (LR): 2e-5
Max Sequence Length: 512
Warmup Ratio: 0.1 (10% of total training steps)
Max Positives per Query: 8

Provide a training loss curve with at least 5 data points to illustrate the training process.



Report

Q1: Retriever & Reranker Tuning (5%)

Reranker Training Process (2.5%): Clearly describe how the training data is constructed

- Stage 1: **Hard Negative Generation**

This stage's purpose is to pair each query from **train.txt with relevant "hard negatives."**

Load Retriever: The script first **loads the first-stage retrieval model** from the ./retriever path.

Recall Candidates: For each query in train.txt, the **retriever searches the FAISS index to find the top 50** (can be modified) most relevant candidate passages.

Label Samples: The script then compares the retrieved passage IDs against the qrels.txt (ground truth) file. Among the 50 recalled candidates:

> **If a passage ID is in the qrels gold standard, it is marked with label 1 (positive).**
>
> **If it is not, it is marked with label 0 (negative).**

These are Hard Negatives because they are passages the **retriever considered relevant (and thus recalled) but are incorrect**.

This new dataset contains the query, the top 50 candidate passages (evidences), and their 0/1 labels (retrieval_labels

Report

Q1: Retriever & Reranker Tuning (5%)

Reranker Training Process (2.5%): Clearly describe how the training data is constructed

- Stage 2: **Reranker Training**

This stage is executed by reranker.py, which reads the data from stage to create the final training pairs.

Anchor: data['**rewrite**'], which is the original query string.

Positive Sampling: For each query, the script first attempts to read the **official "golden" positive passage from qrels.txt and creates an InputExample with a label of 1**. If the positive passage is not found in qrels (or is missing from the corpus.txt), the script falls back to using a passage marked with label 1 from the evidences list (generated in Stage 1) as the positive sample. They are: **(query, positive_passage, 1)**

Negative Sampling: The script iterates through the evidences list (the top 50 candidates recalled by the retriever). **It selects passages that are marked with label 0 (Hard Negatives).** Each query (and its corresponding positive) is paired with a maximum of 7 hard negatives(can be modified). They are: **(query, hard_negative_passage, 0).**

Ultimately, the training data fed into the Reranker model consists of tuples in the format **(query, positive_passage, 1) and (query, hard_negative_passage, 0).**

Report

Q1: Retriever & Reranker Tuning (5%)

Reranker Training Process (2.5%): Specify and explain the loss function used for training.

We have a **binary classification task**, where the goal is to predict a continuous score that will be mapped to the labels 0 or 1. Hence, I use the **CrossEncoder** from the sentence-transformers library

In this configuration, the default loss function used by sentence-transformers is **Binary Cross-Entropy with Logits Loss** (BCEWithLogitsLoss).

This loss function internally **applies a Sigmoid function** to the model's single logit output (converting it to a probability between 0 and 1) and then calculates the **Binary Cross-Entropy** against the ground truth labels (0 or 1).

$$\text{Loss}(x, y) = -[y \cdot \log(\sigma(x)) + (1 - y) \cdot \log(1 - \sigma(x))]$$

Report

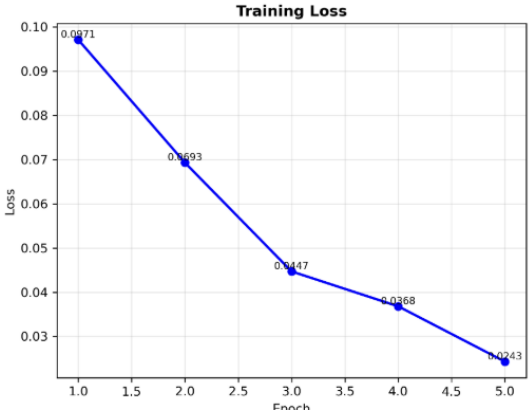
Q1: Retriever & Reranker Tuning (5%)

Reranker Training Process (2.5%):

List the hyperparameters adopted in your experiments.

- Epochs: 5
- Batch Size (batch_size): 32
- Learning Rate (lr): 2e-5
- Warmup Ratio (warmup_ratio): 0.1 (10% of total training steps)
- Max Sequence Length (max_length): 384
- Hard Negative Generation (top_k): 50 (Recall 50 candidates in Stage 1)
- Max Negatives per Query (max_negs): 7 (Max 7 negatives per positive in Stage 2)

Provide a training loss curve with at least 5 data points to illustrate the training process.



Report

Q2: Prompt Optimization (3%)

- Provide a detailed explanation of how you designed your prompt. (1.5%)

The System Prompt's job is to set the AI's overall "core rules." The design includes three critical elements:

1. **"You are a helpful and precise question-answering assistant. "**
This line instructs the model o avoid vague or overly verbose answers, which is very important for a Q&A task.

2. **"You must use ONLY the provided context to answer the user's query."**
This is the most critical instruction in a RAG pipeline. It explicitly forbids the model from using its internal, pre-trained knowledge (which might be outdated or irrelevant). This rule is designed to prevent "hallucination" and forces all answers to be "grounded" in the context you provide.

3. **"If the answer is not contained within the context, respond with 'CANNOTANSWER'"**
This is the specific execution plan for the "Core Constraint" above. Without this, if the model can't find an answer, it might try to guess or synthesize one from its internal knowledge. (CANNOTANSWER) is essential for later evaluating the model's ability to judge "answerability"

Report

Q2: Prompt Optimization (3%)

- Provide a detailed explanation of how you designed your prompt. (1.5%) (cont.)

The User Prompt provides the specific context and query for the task and triggers the model to generate the answer.

1. **context_str = "\n".join([f"Context {i+1}: {c}" for i, c in enumerate(context_list)])**

Instead of just "dumping" all the retrieved documents together, you are numbering them (Context 1:, Context 2:...). It's easier for the model to differentiate between distinct sources of information.

2. **"Based strictly on the context provided above, please answer the following query."**

This line repeats the "Core Constraint" from the system prompt. This is a very effective technique. In long prompts, models can sometimes pay more attention to the "nearest" instructions. By re-stating the strictly on the context rule after providing all the context and before asking the query, you significantly increase the probability that the model will follow this rule.

For the Parsing Function (parse_generated_answer)

1. **pred_ans.split("Answer:")[-1]**

The model might generate some text before the Answer: (if it doesn't perfectly follow the priming) or that it might copy the Answer: label. Taking the content after the last instance of Answer helps to handle the "dirty" data.

2. **unwanted_prefix = "assistant\n<think>\n\n</think>\n\n"**

The model include chain-of-thought (<think>...</think>) or role tags (assistant) in the output. Removing these in the parsing logic shows excellent foresight for handling the "dirty" data that can occur in real-world deployment.

Report

Q2: Prompt Optimization (3%)

- Present at least three different prompts you experimented with, along with their inference performance, to demonstrate the effectiveness of your optimization approach. (1.5%)

```python
from typing import List
import re

def get_inference_system_prompt() -> str:
    """get system prompt for generation"""
    prompt = ""
    return prompt

def get_inference_user_prompt(query : str, context_list : List[str]) -> str:
    """Create the user prompt for generation given a query and a list of cont
    prompt = f""""""
    return prompt

def parse_generated_answer(pred_ans: str) -> str:
    """Extract the actual answer from the model's generated text."""
    parsed_ans = pred_ans
    return parsed_ans
```

Bi-Encoder CosSim: 0.0345

As **no instructions for the LLM**, it does not know what to do. Hence, we showed that System prompt and User prompt has a HUGE impact to LLM.

```python
def get_inference_system_prompt() -> str:
    """get system prompt for generation"""
    prompt = (
        "You are a helpful and precise question-answering assistant. "
        "You must use ONLY the provided context to answer the user's query. "
        )
    return prompt

def get_inference_user_prompt(query : str, context_list : List[str]) -> str:
    context_str = "\n".join([f"Context {i+1}: {c}" for i, c in enumerate(context_list)])
    prompt = f"""Here are the relevant passages retrieved from the knowledge base:
        --- CONTEXT ---
        {context_str}
        --- END CONTEXT ---
        Based strictly on the context provided above, please answer the following query.
        --- QUERY ---
        {query}
        --- END QUERY ---
        Answer:"""
    return prompt

def parse_generated_answer(pred_ans: str) -> str:
    """Extract the actual answer from the model's generated text."""
    if "Answer:" in pred_ans:
        parsed_ans = pred_ans.split("Answer:")[-1]
    else:
        parsed_ans = pred_ans
    return parsed_ans
```

Bi-Encoder CosSim: 0.3765

LLM is **asked to answer the query from given context**. Context are concatenate together.

```python
def get_inference_system_prompt() -> str:
    """get system prompt for generation"""
    prompt = (
        "You are a helpful and precise question-answering assistant. "
        "You must use ONLY the provided context to answer the user's query. "
        "If the answer is not contained within the context, respond with 'CANNOTANSWER'"
        )
    return prompt

def get_inference_user_prompt(query : str, context_list : List[str]) -> str:
    context_str = "\n".join([f"Context {i+1}: {c}" for i, c in enumerate(context_list)])
    prompt = f"""Here are the relevant passages retrieved from the knowledge base:
        --- CONTEXT ---
        {context_str}
        --- END CONTEXT ---
        Based strictly on the context provided above, please answer the following query.
        --- QUERY ---
        {query}
        --- END QUERY ---
        Answer:"""
    return prompt
def parse_generated_answer(pred_ans: str) -> str:
    """Extract the actual answer from the model's generated text."""
    if "Answer:" in pred_ans:
        parsed_ans = pred_ans.split("Answer:")[-1]
    else:
        parsed_ans = pred_ans
    parsed_ans = parsed_ans.strip()
    unwanted_prefix = "assistant\n<think>\n\n</think>\n\n"
    if parsed_ans.startswith(unwanted_prefix):
        parsed_ans = parsed_ans[len(unwanted_prefix):]
    return parsed_ans.strip()
```

Bi-Encoder CosSim: 0.4104

LLM **respond "CANNOTANSWER"** if not able to answer from content.

Unwanted prefix were trimmed for better quality of answering.

Report Q3: Additional Analysis (2%)

- Show any interesting analysis using plots or experimental data obtained during inference. (2%)
- Observation 1: Wrong Retrieve, but quite OK answering

Recall@10 (MNRL)	0.8340	✅ The Retriever using correct loss function
Recall@10 (MNRL+TML)	0.0000	❌ The Retriever using INCORRECT loss function

MultipleNegativesRankingLoss (MNRL) has an objective to correctly classify the positive sample from a set of negatives. Whereas, **TripletLoss (TripletMarginLoss)** has an objective to ensure a relative distance is maintained. Originally, I thought two will need to be combined to make positive closer and negative further. However, **MNRL** is essentially a more modern, efficient, and powerful version of a triplet-style loss. It's **already doing the job** of pushing negatives away, as **two gradients are likely not aligned and it retrieved NOTHING**.

MRR@10 (with Rerank)	0.0000	❌ Nothing correct is retrieved so nothing correct is reranked
Bi-Encoder CosSim	**0.2675**	✅ Topically related or Accidental Correctness

Reranker can do nothing since retriever gave it wrong answer. But, surprisingly, the **LLM** while receiving incorrect context, can **still have partial score of Bi-Encoder CosSim**. Perhaps, due to two possibilities:

1. Retriever retrieves wrong but topical related content. **2. The LLM itself can answer it right.**

Report Q3: Additional Analysis (2%)

- Show any interesting analysis using plots or experimental data obtained during inference. (2%)
- Observation 2: Distribution Mismatch of Retriever and Reranker

MRR@10 (without Rerank)	0.6388	✅ The Retriever's initial ranking is OK.
MRR@10 (with Rerank)	0.5689(7.0% Drop)	❌ The Reranker is pushing the correct answer down in the list.

When I first trained the reranker, **the training data I chose for reranker was from 'evidences' in train.txt**. The data above shows that while the necessary document is present (Recall@10 is high), the **reranker's new ordering is less accurate than the retriever's original simple score**.

This is probably due to the misalignment of negative samples. **As the negative sample during testing is from those retrieved, instead of evidences labeled in train.txt.**

Therefore, I altered the **training data for reranker into those retrieved from retriever. (mentioned in Q1)** After the reranker were trained based on new training data. The MRR@10 increased as it should do.

MRR@10 (without Rerank)	0.6388	✅ The Retriever's initial ranking is OK.
MRR@10 (refined Rerank)	0.7536(11.5% up)	✅ The Reranker is pushing the correct answer up in the list.

Report Q3: Additional Analysis (2%)

- Show any interesting analysis using plots or experimental data obtained during inference. (2%)
- Observation 3: Inspired by RL bonus homework:

I tried different top_m in the inference batch for additional analysis. Originally, in my perspective, larger m should help to find more context, and hence, yield a better result.

Bi-Encoder CosSim(m = 2)	0.4188	Smaller m yields better result
Bi-Encoder CosSim(m = 3)	0.4104	**The inference batch selection by this homework**
Bi-Encoder CosSim(m = 4)	0.4054	Larger m yields worse result
Bi-Encoder CosSim(m = 5)	0.3969	Larger m yields worse result

However, according to the result, lower m yields a better result, which may be due to less interfere of LLM by non-relevant prompts, as MRR@10 already yields > 0.5 result (where a single 0.5 before averaging stands for result occur at the second place.)

Report

Bonus: RL in the loop (2%)

- Describe your training method and experimental setting to compare with the original results.

My core method is to train a PPO (Proximal Policy Optimization) Agent to act as a Dynamic Selector for the RAG pipeline. This agent learns to choose the optimal number of top-ranked documents (M) to pass to the Large Language Model (LLM) for answer generation, on a query-by-query basis.

The RL problem is formally defined by the custom Gymnasium environment:

Action Space (A): An integer action from {0,1,2,3,4}, which corresponds to selecting **M=1 to M=5** passages.

Observation Space (S): 4-Dimensional Feature Vector derived from the Reranker scores of the Top-10 retrieved documents. This state attempts to capture the *quality and confidence* of the retrieved context:

1. **Top-1 Score** 2. **Top-1 / Top-2 Score Gap** 3. **Mean** of Top-10 scores 4. **Standard Deviation** of Top-10 scores

Reward Function (R): The reward is the **Sentence Similarity** (Cosine Similarity) between the LLM's generated answer and the ground-truth answer, calculated using the all-MiniLM-L6-v2 Sentence Scorer.

Agent: The PPO agent uses an **MlpPolicy** to map the 4-D observation state to the optimal action (the choice of M).

Report

Bonus: RL in the loop (2%)

- Describe your training method and experimental setting to compare with the original results.(cont.)

An additional inference.py is needed, as inference_batch.py did not consider the PPO result and did not support a fluctuating top_m.

After inference, we get the result:

Bi-Encoder CosSim(m = 3)	0.4104	**The top_m selection by this homework**
Bi-Encoder CosSim(RL)	0.4247	✅ Yields a better result than a fixed top_m chosen

Not a surprising due to the additional observation 3 tells that a fixed m = 3 is not the best.

But, an improvement after RL still stands for a promising result.

Reference

- https://stable-baselines3.readthedocs.io/en/master/
- https://gymnasium.farama.org/index.html
- https://arxiv.org/abs/2406.06475
- https://github.com/facebookresearch/faiss
- https://aclanthology.org/W04-1013.pdf
- https://en.wikipedia.org/wiki/Mean_reciprocal_rank
- https://sbert.net/examples/cross_encoder/applications/README.html

Discussed with Gemini to build retriever.py/reranker.py/reinforcement.py